SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5116

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Assessment Tool Weightage : 20%

QUESTIONS: **Total Marks: 50**

Annexure A

Scenario-Based

Code of Conduct:

I J.Vishwanth (192573030) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

J.Vishwanth

Signature of the student:

Annexure A
Scenario-Based

1. Secure Data Transmission using Block Cipher

A company is transmitting repeated financial data blocks over a network. Initially, they used ECB mode and noticed patterns in ciphertext.

A. Explain why this issue occurs.

A block cipher like AES or DES encrypts data in fixed size blocks (128 bits in AES and 64 bits in DES). The mode of operation tells us how the blocks of a long message are encrypted one after the other. ECB means Electronic Codebook mode. In this mode the message is divided into blocks and each block is encrypted separately using the same key.

$$C_i = E(K, P_i)$$

There is no IV and no chaining, so one block does not depend on any other block. Because of this, ECB always gives the same output for the same input. If two plaintext blocks are equal, their ciphertext blocks are also equal. Financial data has a lot of repetition, for example the same account header, the same field names, the same amount or the same padding bytes. These repeated blocks produce repeated ciphertext blocks, so the company sees patterns in the ciphertext.

Block	Plaintext (before encryption)	Ciphertext (ECB)
1	ACCOUNT-BALANCE	X7A9
2	10000 RUPEES	K2M4
3	ACCOUNT-BALANCE	X7A9 (same as block 1)
4	10000 RUPEES	K2M4 (same as block 2)

This causes the following problems:

- An attacker can see which blocks are equal without knowing the key, so the structure of the data is leaked and frequency analysis becomes possible.
- The blocks are independent, so an attacker can copy, delete, repeat or swap blocks (for example a block that holds an amount) and the receiver will not notice.
- If the attacker knows one plaintext block, he can find that block at every place where the same ciphertext block appears.

So ECB does not hide data patterns and it is not suitable for real financial data.

B. Suggest a more secure block cipher mode and justify your choice based on the scenario.

I suggest CBC (Cipher Block Chaining) mode with AES and a new random IV for every message. In CBC, a random Initialization Vector (IV) is generated first. Each plaintext block is XORed with the previous ciphertext block before it is encrypted, and for the first block the IV is used.

$$C_1 = E(K, P_1 \text{ XOR } IV) \qquad C_i = E(K, P_i \text{ XOR } C_{i-1})$$

$$P_1 = D(K, C_1) \text{ XOR } IV \qquad P_i = D(K, C_i) \text{ XOR } C_{i-1}$$

Why CBC suits this scenario:

- Each ciphertext block depends on all the blocks before it, so even if the financial blocks are identical the ciphertext blocks are different. The patterns seen in ECB disappear.
- The IV is random for every transmission, so the same message sent twice gives a different ciphertext. The IV is not secret and can be sent along with the ciphertext.
- If a block is changed or swapped, the next block gets corrupted after decryption, so the cut and paste attacks that work on ECB are much harder.
- It uses the same cipher and the same key, so ECB can be replaced by CBC in the existing system without big changes.

CBC also has some limits. Encryption is sequential, the last block needs padding, and CBC alone does not check integrity. So an HMAC should be added, or AES-GCM can be used.

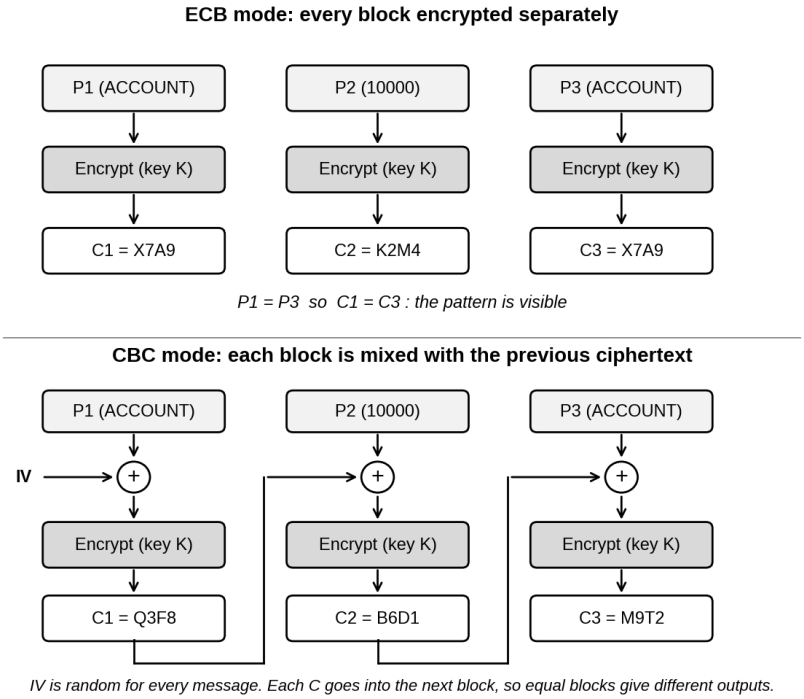


Figure 1: ECB and CBC encryption of the same blocks (+ means XOR)

Feature	ECB	CBC
Same plaintext blocks	Same ciphertext blocks	Different ciphertext blocks
IV needed	No	Yes (random)
Pattern leakage	High	Very low
Block swap or replay	Easy for attacker	Difficult, gets detected
Suitable for financial data	No	Yes

Other modes like CTR and GCM can also be used. CTR encrypts a counter and XORs it with the data, which is fast and also hides patterns, but a counter must never repeat with the same key. GCM is CTR with an authentication tag, so it gives encryption and integrity together and is the best modern choice. CFB and OFB turn the block cipher into a stream cipher but they give no authentication.

Recommendations for the company:

- Use AES-128 or AES-256 instead of DES, because the 56 bit key of DES is too weak now.
- Generate a fresh random IV for every message with a secure random number generator and send it with the ciphertext.
- Add integrity protection by using HMAC-SHA256 on the IV and ciphertext, or use AES-GCM, because an attacker can modify plain CBC data without the receiver knowing.
- Send the data over a secure channel like TLS and change the key regularly.

Conclusion: ECB leaks patterns because equal plaintext blocks give equal ciphertext blocks. CBC with a random IV removes this problem, and adding an HMAC or using GCM also protects the financial data from tampering.

4. Public Key Cryptography for Email Security

An organization plans to implement RSA for secure email communication.

A. Describe how RSA can be used for encryption and decryption in this scenario.

RSA is a public key (asymmetric) algorithm. Every user has a public key, which is shared with everyone, and a private key, which is kept secret. Its security depends on the fact that it is very hard to factorize a large number n into its two prime factors.

Key generation (done by the receiver, for example Bob):

1. Choose two large prime numbers p and q.
2. Calculate $n = p \times q$.
3. Calculate $\varphi(n) = (p - 1)(q - 1)$.
4. Choose e such that $1 < e < \varphi(n)$ and $\gcd(e, \varphi(n)) = 1$.
5. Find d such that $(e \times d) \bmod \varphi(n) = 1$.

The public key is (e, n) and the private key is (d, n).

Use in email:

- Encryption: Alice takes Bob's public key and encrypts the email as $C = M^e \bmod n$. Only Bob has the private key, so only he can read it. This gives confidentiality.
- Decryption: Bob gets the email back using his private key, $M = C^d \bmod n$.
- Digital signature: Alice makes a hash of the email and encrypts the hash with her private key. Bob checks it with Alice's public key. This proves that the email came from Alice and was not changed.

Small example: let p = 3 and q = 11, so n = 33 and $\varphi(n) = 20$. Take e = 7, then d = 3 because $7 \times 3 = 21$ and $21 \bmod 20 = 1$. For the message M = 4, the ciphertext is $C = 4^7 \bmod 33 = 16384 \bmod 33 = 16$. For decryption, $M = 16^3 \bmod 33 = 4096 \bmod 33 = 4$, which is the original message.

In real email systems like S/MIME and PGP, RSA is not used for the whole message because it is slow and can only encrypt data smaller than the key size. A hybrid method is used. A random AES session key encrypts the email body and RSA encrypts only this small session key. Bob first decrypts the session key with his private key and then uses it to decrypt the email. Keys of 2048 bits or more are used.

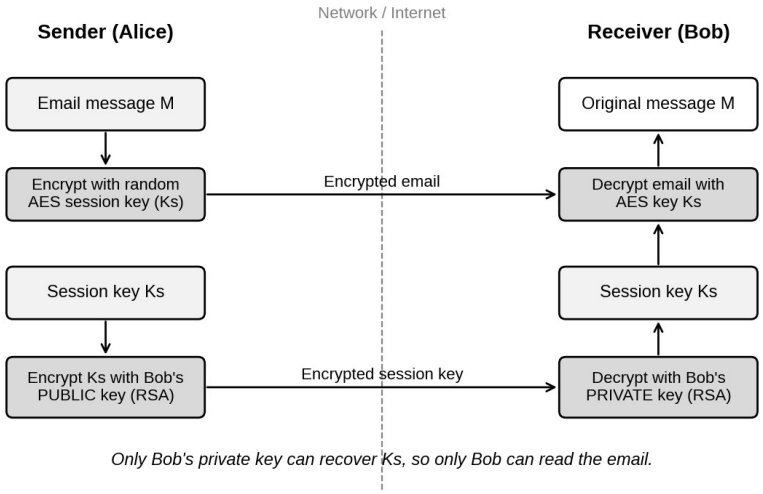


Figure 2: Secure email using RSA with an AES session key

Steps for sending a secure email:

Step	Who	Action
1	Bob	Generates his RSA key pair and publishes the public key in a certificate
2	Alice	Gets Bob's public key and checks that the certificate is valid
3	Alice	Creates a random AES session key Ks and encrypts the email with it
4	Alice	Encrypts Ks with Bob's public key: $C = Ks^e \bmod n$
5	Alice	Hashes the email (SHA-256) and signs the hash with her private key
6	Alice	Sends the encrypted email, the encrypted Ks and the signature to Bob
7	Bob	Decrypts Ks with his private key: $Ks = C^d \bmod n$
8	Bob	Decrypts the email using Ks and verifies Alice's signature with her public key

RSA is used for email because the sender and receiver do not need to share a secret key in advance. Anyone can encrypt with the public key but only the owner of the private key can decrypt. RSA also gives authentication and non repudiation, which means the sender cannot deny sending the email.

Point	Symmetric (AES)	RSA (public key)
Keys	One shared secret key	Public key and private key
Speed	Very fast	Slow
Key distribution	Difficult, key must be shared secretly	Easy, public key can be shared openly
Used in email for	Encrypting the message body	Encrypting the session key and signing

Plain RSA is not secure by itself, so padding like OAEP is used for encryption and PSS for signatures. Small keys of 512 or 1024 bits can be broken, so at least 2048 bits is recommended.

B. Explain how key distribution and management play a role in maintaining security.

RSA is mathematically strong but the system is secure only if the keys are handled properly. If a wrong public key is used or the private key is leaked, the encryption is of no use. Key distribution and management help in the following ways:

- Public key distribution: the public key must really belong to the person. If an attacker gives a fake public key, he can read the emails (man in the middle attack). To avoid this, public keys are shared using digital certificates (X.509) signed by a trusted Certificate Authority (CA) in a Public Key Infrastructure (PKI). PGP uses a web of trust and key servers.
- Private key protection: the private key must never be shared. It is kept in an encrypted key store, smart card or Hardware Security Module (HSM) and protected with a strong passphrase.
- Secure key generation: keys are generated with a good random number generator and large primes (2048 bits or more) so that they cannot be guessed or factorized.
- Key lifetime and renewal: keys and certificates have an expiry date and are changed regularly, which limits the damage if a key is stolen.
- Revocation: if a private key is lost or compromised, the certificate is revoked using a CRL or OCSP so that nobody trusts that key any more.
- Backup and recovery: private keys are backed up safely (key escrow) so that old encrypted emails can still be opened if the user loses the key.

Conclusion: RSA gives confidentiality, authentication and integrity for email, and proper key distribution (certificates and CA) and key management (protection, renewal, revocation, backup) make sure that these benefits are not lost.